Bases de Datos Relacionales

DDL

Data Definition Languaje + Triggers



DDL. Data Definition Languaje



El **DDL** (Data Definition Language o Lenguaje de Definición de Datos) en SQL incluye todos los comandos y sentencias utilizados para **crear, modificar o eliminar la estructura de los objetos de una base de datos** (tablas, vistas, índices, esquemas y restricciones), pero no los datos que contienen.

IF EXISTS | IF NOT EXISTS		CREATE	ALTER	DROP	RENAME	TRUNCATE
		Crear	Modificar	Eliminar	Cambiar Nombre	Vaciar
UF03	DATABASE	x	x	x		
	[TEMPORARY] TABLE	x	x	x	x	x
	TRIGGER	x		x		
UF04	VIEW	x		x		
UF05	INDEX	x		x		
	PROCEDURE	x		x		
	FUNCTION	x		x		
	EVENT	x	x	x		

CREATE ... [IF NOT EXISTS]



Permite crear nuevos objetos en la base de datos (por ejemplo, Bases de Datos, Tablas, Triggers (disparadores), Índices, etc).

La cláusula `IF NOT EXISTS` sirve para evitar errores en SQL al intentar crear un objeto que ya existe en la base de datos. Al añadir esta cláusula, se le indica al sistema que solo cree el objeto si no se encuentra en el sistema, y que si ya existe, simplemente ignore la instrucción de forma segura y continúe con el resto del script sin fallar. Ejemplo:

```
CREATE TABLE IF NOT EXISTS usuarios (
   id_usuario INT PRIMARY KEY,
   usuario VARCHAR(50) comment 'una letra mayúsc., una  minúsc, un número y un símbolo'
   email VARCHAR(100)
);
```

`CREATE SCHEMA` es un sinónimo para `CREATE DATABASE` en MySQL

```
CREATE DATABASE IF NOT EXISTS nomina
    DEFAULT CHARACTER SET utf8mb4
--  Juego de caracteres predeterminado. utf8mb4 soporta todos los caracteres Unicode,
--  incluidos tildes, eñes y emojis.
    DEFAULT COLLATE utf8mb4_general_ci
--  cotejamiento (reglas de comparación y ordenamiento). _ci significa case-insensitive.
    DEFAULT ENCRYPTION = 'N';
--  Permite activar ('Y') o desactivar ('N') el cifrado de las tablas
```

ALTER TABLE [IF EXISTS]



La instrucción `ALTER TABLE` es una de las más complejas y potentes, ya que permite modificar completamente la estructura de una tabla existente.

Las opciones principales alter table se agrupan en las siguientes categorías de operaciones:

Columnas

- ADD [COLUMN] FIRST | AFTER
- DROP COLUMN
- CHANGE COLUMN

```
ALTER TABLE
 ADD COLUMN
  iva DECIMAL(5,2)
 AFTER precio
```

Claves

- ADD PRIMARY KEY
- DROP PRIMARY KEY
- ADD FOREIGN KEY
- DROP FOREIGN KEY
- ADD UNIQUE
- DROP UNIQUE

```
ALTER TABLE usuarios
 ADD CONSTRAINT
 uq_email
 UNIQUE (email);
```

Restricciones

- ADD CONSTRAINT
- DROP CONSTRAINT nombre_restriccion

```
ALTER TABLE ventas
 ADD CONSTRAINT c1
 CHECK (precio > 0);
```

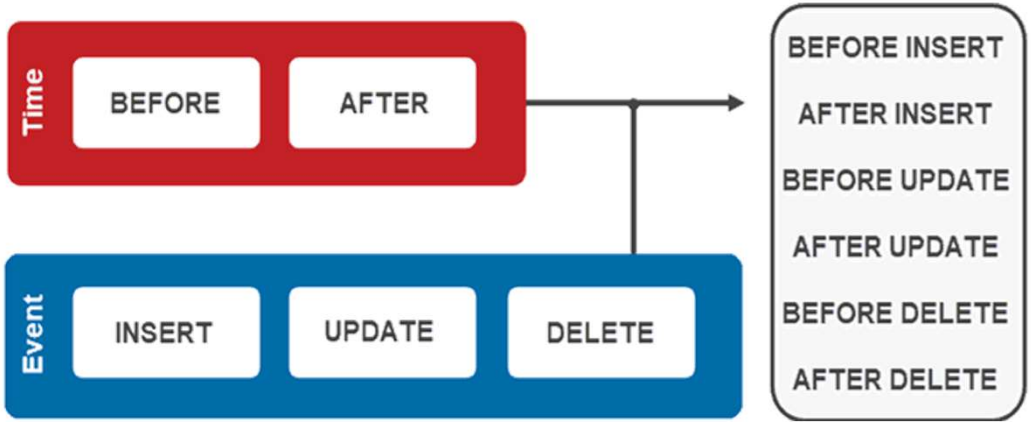
Es una buena práctica ponerle un nombre a las restricciones para poder identificarlas o borrarla fácilmente en el futuro

Diferencia crítica entre DROP, TRUNCATE y DELETE

La diferencia crítica radica en qué nivel de la base de datos operan (Estructura vs. Datos), la velocidad de ejecución y si se pueden deshacer o no.

	DROP	TRUNCATE	DELETE
Definición	Borra tanto la estructura de la tabla como todo lo que contiene (datos, índices, privilegios, restricciones)	MySQL destruye la tabla y la vuelve a crear vacía de forma instantánea. Por eso que el contador de los campos AUTO_INCREMENT vuelve a empezar desde el 1.	MySQL va fila por fila analizando cuáles cumplen la condición del WHERE (si no pones WHERE, va fila por fila borrándolo todo).
Categoría	**DDL**	**DDL**	**DML**
¿Qué elimina?	La tabla entera y sus datos	Solo los datos (vacía la tabla)	Filas específicas o todas
¿Permite WHERE?	No	No	**Sí** (puedes filtrar)
Velocidad	Instantáneo	Muy rápido	Lento (fila por fila)
Auto-increment	Se destruye	**Se reinicia a 1**	Mantiene el conteo actual
¿Se puede deshacer?	No (con excepciones de backup)	No	**Sí** (con ROLLBACK)
Triggers	No activa nada	No activa nada	**Activa los triggers** BEFORE/AFTER DELETE

Triggers



(Disparadores)

https://dev.mysql.com/doc/refman/8.4/en/trigger-syntax.html

Disparadores [`Triggers`]



Un trigger o disparador es un script que consiste en una secuencia de instrucciones que se asocian a una tabla que se ejecutan automáticamente cuando se realizan determinadas operaciones en la misma: **añadir, actualizar o eliminar registros**.

Usos prácticos de los disparadores:

- ✓ Generación de históricos.
- ✓ Realización de Auditorías sobre operaciones clave (Audit Trails).
- ✓ Generación de Alertas.
- ✓ Bloqueo de inserción para datos incorrectos o transacciones inválidas.
- ✓ Implementar reglas de consistencia de datos no provistas por el modelo relacional.

Tipos de Triggers



Triggers de Inserción (INSERT): se ejecutan automáticamente **antes** o **después** de insertar una nueva fila en una tabla. Es útil para tareas como generar identificadores únicos o rellenar valores predeterminados.

Triggers de Actualización (UPDATE): se desencadenan cuando se actualiza una fila de una tabla. Se pueden usar para aplicar restricciones, actualizar datos relacionados o realizar acciones adicionales basadas en cambios.

Triggers de Eliminación (DELETE):
se activan cuando se elimina una fila de una tabla. Se pueden usar para aplicar la integridad referencial, realizar acciones de limpieza o guardar datos históricos .

Uso de identificadores de correlación old y new

Al hacer referencia a los campos de la tabla a la cual se le genera un disparador, el prefijo OLD contiene los valores de la fila afectada antes de ser “modificada”, mientras que NEW contiene los valores de la fila afectada después de ser modificada.

new almacena la lista de los valores que se van a insertar o actualizar en la tabla.

old almacena la lista de los valores antiguos que se actualizan con los nuevos valores, o que se eliminarán de la base de datos.



DML / Pseudo Records	Insert	Update	Delete
New	✓	✓	✗
Old	✗	✓	✓

Sintaxis para la creación de un Trigger



PARTE DE LA SENTENCIA	EXPLICACIÓN
`CREATE TRIGGER <nombre del trigger>`	Crea el disparador con el nombre especificado
`{ BEFORE | AFTER }`	Indicamos en qué momento se ejecutará el Trigger.
`{ INSERT | UPDATE | DELETE }`	Indicaremos para cual comando SQL se invocará el Trigger.
`ON <nombre de tabla>`	Indica la tabla (o vista) asociada al disparador.
`FOR EACH ROW ...`	En MySQL el cuerpo del trigger se ejecuta individualmente para cada una de las filas de la tabla.

La cláusula **DELIMITER**



Para definir un objeto almacenado con instrucciones SQL (trigger, function o Stored Procedure) es necesario modificar temporalmente el carácter separador que se utiliza para delimitar las sentencias SQL.

El carácter separador que se utiliza por defecto en SQL es el punto y coma (;).

Es posible utilizar cualquier carácter (o conjunto de caracteres) que no sea ambiguo.

En el siguiente ejemplo se utilizan los caracteres **$$** para delimitar las instrucciones SQL.:

```
DELIMITER $$
     Cuerpo del disparador, función o procedimiento
DELIMITER ;
```

Ejemplo Triggers (1)



```
DELIMITER $$
  CREATE TRIGGER auditoria_empleados_insertar
  AFTER INSERT ON Empleados
  FOR EACH ROW
    BEGIN
      INSERT INTO empleados_auditoria
         (id_empleado,
          fecha_cambio,
          tipo_cambio,
          responsable,
          datos_cambio)
      VALUES
         (NEW.id_empleado,
          NOW(),
          'INSERCION',
          USER(),
          CONCAT('Nuevo empleado: ', NEW.nombre, ' ', NEW.apellido)
         );
  END;
DELIMITER ;
```

Lanzar un error personalizado dentro de un disparador (trigger)



En MySQL, la instrucción **`SIGNAL SQLSTATE '45000'`** se utiliza para lanzar un error personalizado de forma manual dentro de un disparador (*trigger*), función o procedimiento almacenado.

Al ejecutarse, interrumpe inmediatamente la operación actual (como un **`INSERT`** o **`UPDATE`**) y evita que se guarden datos incorrectos en la base de datos.

- **`SIGNAL`** : Es el comando utilizado para "lanzar" el error (excepción). Es similar al **`throw`** o **`raise`** en otros lenguajes de programación.
- **`SQLSTATE '45000'`** : Es un código estándar de 5 caracteres que significa "error definido por el usuario no controlado". El valor **`'45000'`** es un código genérico reservado explícitamente por el estándar SQL para que los desarrolladores definan sus propios errores personalizados sin entrar en conflicto con los códigos de error internos del sistema.
- **`SET MESSAGE_TEXT = ...:`** Es el mensaje de texto legible que verá el usuario o la aplicación que intenta realizar la consulta (en este caso, explicando que la cantidad debe ser mayor a cero).

Ejemplo Triggers (2)



```
DELIMITER $$

CREATE TRIGGER tr_ventas
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
-- Verificar que la cantidad vendida sea mayor que cero
IF NEW.cantidad <= 0 THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT = 'La cantidad vendida debe ser mayor que cero';
END IF

DELIMITER ;
```